

Optimización de Rutas mediante Metaheurísticas Bio-Inspiradas Aceleradas por GPU

Algoritmo de Colonia de Hormigas (ACO)
implementado con **Numba CUDA** sobre GPU NVIDIA Tesla T4

Guillermo Andres De Mendoza Corrales
Proyecto de Metaheurísticas Bio-Inspiradas

Implementación ejecutada en Google Colaboratory
Runtime: GPU T4 — Python 3.12 — Numba 0.6x — NumPy

Índice

1. Introducción	2
1.1. Motivación biológica	2
2. Objetivo	2
3. Parámetros del algoritmo	3
4. Espacio del experimento	3
4.1. Matriz del entorno	3
4.2. Matriz de feromonas	4
4.3. Heurística	4
5. Selección probabilística de movimientos	4
6. Ciclo de vida del algoritmo	5
6.1. Backtracking	5
6.2. Límite de pasos adaptativo	5
7. Implementación GPU con Numba CUDA	5
7.1. Entorno de ejecución	5
7.2. Justificación del uso de Matplotlib en lugar de tkinter	6
7.3. Los cuatro kernels CUDA	6
7.3.1. Configuración del grid CUDA	6
7.3.2. Gestión eficiente de memoria GPU	6
7.3.3. Depósito paralelo con operaciones atómicas	7
8. Visualización	7
8.1. Mapa inicial	7
8.2. Curva de convergencia	7
8.3. Animación	7
8.4. Resultado final	8
9. Resultados obtenidos	8
9.1. Nota sobre la advertencia de ocupación GPU	8
10. Conclusiones	9

1. Introducción

Una **metaheurística** es un método de búsqueda de soluciones que no garantiza encontrar el óptimo global, pero produce soluciones de alta calidad en tiempo razonable. Las metaheurísticas **bio-inspiradas** imitan procesos naturales: evolución genética, comportamiento de enjambres, sistemas inmunológicos, entre otros.

El presente proyecto implementa el **Algoritmo de Colonia de Hormigas** (*Ant Colony Optimization*, ACO), inspirado en el comportamiento colectivo de colonias de hormigas reales para encontrar rutas eficientes hacia fuentes de alimento.

1.1. Motivación biológica

Las hormigas reales son individualmente simples: no poseen un mapa del entorno ni coordinación central. Sin embargo, la colonia en conjunto logra encontrar rutas óptimas mediante un mecanismo denominado **estigmergia**: comunicación indirecta a través del entorno.

El proceso funciona de la siguiente manera:

1. Una hormiga sale del hormiguero y explora el entorno de forma **aleatoria**.
2. Al encontrar comida, regresa depositando una sustancia química llamada **feromona** a lo largo de su camino.
3. Otras hormigas detectan la feromona y tienden a seguir ese rastro.
4. Las rutas más cortas reciben más feromonas (se recorren más veces en el mismo tiempo) y son progresivamente preferidas.
5. Las rutas largas pierden feromona por **evaporación** y se abandonan gradualmente.

El resultado es un camino óptimo emergente **sin ninguna coordinación central**, fenómeno de *inteligencia de enjambre*.

2. Objetivo

Implementar el algoritmo ACO utilizando **GPU** para potenciar los cálculos vectoriales y matriciales del experimento. El sistema debe encontrar la ruta más corta (o de menor costo) entre un nodo origen A y un nodo destino B en una matriz 2D, simulando el comportamiento de una colonia de hormigas que depositan feromonas.

Los elementos principales del experimento son:

- **Hormiga**: agente que explora el espacio y construye soluciones paso a paso.
- **Camino de feromonas**: valores numéricos que influyen las decisiones de movimiento.
- **Heurística**: conocimiento del problema ($\eta = 1/\text{distancia}$) que guía las decisiones.

- **Movimiento probabilístico:** decisión de movimiento influenciada por feromonas y heurística.
- **Evaporación:** reducción gradual de feromonas para eliminar caminos subóptimos.
- **Madriguera:** punto de origen de todas las hormigas.
- **Fuente de comida:** destino objetivo.

3. Parámetros del algoritmo

Símbolo	Nombre	Rango usado	Descripción
N	Hormigas	[20, 60]	Cantidad de agentes por iteración
M	Iteraciones	[80, 150]	Ciclos totales del algoritmo
X	Columnas	[15, 25]	Ancho de la matriz espacial
Y	Filas	[15, 25]	Alto de la matriz espacial
τ	Feromona	$\tau_0 \in [0,5, 1,5]$	Nivel de feromona por celda
ρ	Evaporación	[0,1, 0,4]	Tasa de disminución por iteración
α	Peso feromona	[0,5, 2,0]	Importancia relativa de τ
β	Peso heurística	[1,0, 4,0]	Importancia relativa de η
Q	Depósito	[50, 200]	Constante de refuerzo de feromona
τ_{min}	Piso feromona	10^{-4}	Valor mínimo para evitar extinción

Cuadro 1: Parámetros del experimento ACO. Los valores se generan aleatoriamente en cada ejecución para garantizar variedad experimental.

Los parámetros α , β y Q no aparecen explícitamente en el enunciado original pero son componentes estándar e indispensables de ACO: sin ellos no es posible calcular la fórmula de selección probabilística ni el depósito de feromonas.

4. Espacio del experimento

4.1. Matriz del entorno

Se implementa una matriz de tamaño $X \times Y$ con cuatro tipos de celda:

Valor	Tipo	Descripción
0	FREE	Celda transitable
1	OBSTACLE	Pared — las hormigas no pueden atravesarla
2	ORIGIN	Madriguera (punto de partida)
3	FOOD	Fuente de comida (destino)

Cuadro 2: Tipos de celda en la matriz del entorno.

Los obstáculos se colocan aleatoriamente cubriendo entre el 15 % y el 28 % de las celdas. El origen y el destino se seleccionan en celdas libres con una distancia Manhattan mínima de $\max(X, Y)/2$, garantizando que el experimento sea no trivial.

4.2. Matriz de feromonas

Se mantiene una segunda matriz τ de igual tamaño que el entorno:

- Celdas libres: inicializadas en τ_0 .
- Celdas obstáculo: valor fijo 0,0 (no transitables, no acumulan feromona).

4.3. Heurística

Para cada celda (r, c) , la heurística se calcula una única vez antes del ciclo principal, ya que el destino no cambia:

$$\eta[r, c] = \begin{cases} 0 & \text{si grid}[r, c] = \text{OBSTACLE} \\ \frac{1}{\sqrt{(r - r_f)^2 + (c - c_f)^2}} & \text{en otro caso} \end{cases} \quad (1)$$

donde (r_f, c_f) es la posición del destino. Si la celda es el propio destino, $\eta = 9999$ para evitar división por cero.

5. Selección probabilística de movimientos

En cada paso, cada hormiga elige la siguiente celda según la distribución de probabilidad estándar de ACO:

$$P(i \rightarrow j) = \frac{[\tau(j)]^\alpha \cdot [\eta(j)]^\beta}{\sum_{u \in \mathcal{V}(i)} [\tau(u)]^\alpha \cdot [\eta(u)]^\beta} \quad (2)$$

donde $\mathcal{V}(i)$ es el conjunto de vecinos válidos (transitables y no visitados) de la celda i . La selección se realiza mediante **ruleta estocástica** (*roulette-wheel selection*).

Corrección de precisión flotante: la aritmética de punto flotante de 32 bits puede producir sumas de probabilidades ligeramente distintas de 1.0, lo que causa rechazo en NumPy. Se aplica la corrección:

```
1 probs[-1] = 1.0 - sum(probs[:-1]) # el ultimo elemento absorbe el
  error
2 idx = np.random.choice(len(vecinos), p=probs)
```

Listing 1: Corrección de precisión en ruleta estocástica

6. Ciclo de vida del algoritmo

El flujo de cada iteración es el siguiente:

1. **Pesos GPU**: el kernel `kernel_pesos` precalcula $\tau^\alpha \cdot \eta^\beta$ para cada celda de la matriz en paralelo.
2. **Movimiento (CPU)**: cada hormiga recorre la matriz celda a celda usando los pesos precalculados, con backtracking ante callejones sin salida.
3. **Depósito GPU**: el kernel `kernel_deposito` deposita feromonas de todos los caminos válidos en paralelo:

$$\tau[r, c] += \frac{Q}{|\text{camino}|} \quad \forall (r, c) \in \text{camino} \quad (3)$$

4. **Evaporación GPU**: el kernel `kernel_evaporacion` aplica:

$$\tau[r, c] = \max((1 - \rho) \cdot \tau[r, c], \tau_{\min}) \quad (4)$$

5. **Actualización**: se registra el mejor camino global si alguna hormiga encontró uno más corto.

6.1. Backtracking

La versión estándar de ACO abandona a las hormigas que quedan sin vecinos válidos. Esta implementación aplica **backtracking**: la hormiga retrocede en su historial hasta encontrar la última posición con vecinos disponibles no visitados, y continúa la exploración desde allí. Esto aumenta significativamente la tasa de éxito en mapas con obstáculos densos.

6.2. Límite de pasos adaptativo

El límite de pasos por hormiga se calcula como:

$$\text{max_pasos} = \text{dist_Manhattan}(\text{origen}, \text{destino}) \times 6 \quad (5)$$

Este valor es más justo que un valor fijo arbitrario: da margen suficiente para caminos no directos y lo ajusta automáticamente al tamaño real del problema.

7. Implementación GPU con Numba CUDA

7.1. Entorno de ejecución

La implementación se ejecuta en **Google Colaboratory** con una GPU **NVIDIA Tesla T4** (Compute Capability 7.5, 14.56 GB VRAM), usando la librería **Numba CUDA** para compilar y ejecutar kernels CUDA directamente desde Python.

7.2. Justificación del uso de Matplotlib en lugar de tkinter

El enunciado sugiere `tkinter` para la animación en tiempo real. Google Colab es un entorno *headless* (sin acceso a pantalla física), por lo que `tkinter` no puede abrir ventanas. Se utiliza `matplotlib.animation.FuncAnimation` con renderizado HTML5, que es el equivalente funcional en este entorno: produce una animación interactiva con controles de reproducción embebidos en el notebook.

7.3. Los cuatro kernels CUDA

Kernel	Operación	Paralelismo
<code>kernel_heuristica</code>	$\eta[r, c] = 1/\text{dist}$ para toda la matriz	1 hilo por celda
<code>kernel_evaporacion</code>	$\tau = \max((1 - \rho)\tau, \tau_{\min})$	1 hilo por celda
<code>kernel_deposito</code>	$\tau += Q/ \text{camino} $ para N caminos	1 hilo por camino
<code>kernel_pesos</code>	$\tau^\alpha \cdot \eta^\beta$ para toda la matriz	1 hilo por celda

Cuadro 3: Kernels CUDA implementados y su nivel de paralelismo.

7.3.1. Configuración del grid CUDA

Se usa un grid 2D de bloques de 16×16 hilos (256 hilos/bloque), configuración estándar para matrices 2D en GPUs de Compute Capability 7.x:

```

1 def configurar_grid_cuda(rows, cols, tpb=(16, 16)):
2     bx = (cols + tpb[0] - 1) // tpb[0]
3     by = (rows + tpb[1] - 1) // tpb[1]
4     return (bx, by), tpb

```

Listing 2: Configuración del grid CUDA

7.3.2. Gestión eficiente de memoria GPU

El buffer de feromonas se aloja en GPU **una sola vez** antes del ciclo principal. En cada iteración se reutiliza mediante `copy_to_device` y `copy_to_host`, evitando costosas reasignaciones de memoria en device:

```

1 # Antes del loop: alocar una vez
2 d_pheromone = cuda.to_device(pheromone)
3
4 # Dentro del loop: reutilizar el buffer
5 d_pheromone.copy_to_device(pheromone) # CPU -> GPU
6 kernel_evaporacion[blocks, threads](d_pheromone, ...)
7 cuda.synchronize()
8 d_pheromone.copy_to_host(pheromone) # GPU -> CPU

```

Listing 3: Reutilización del buffer GPU por iteración

7.3.3. Depósito paralelo con operaciones atómicas

El kernel `kernel_deposito` recibe todos los caminos válidos de una iteración empaquetados en un array plano y usa `cuda.atomic.add` para garantizar escrituras concurrentes correctas:

```

1 @cuda.jit
2 def kernel_deposito(pheromone, caminos_flat, caminos_len,
3                     caminos_offset, n_caminos, Q, rows, cols):
4     idx = cuda.grid(1)
5     if idx < n_caminos:
6         length = caminos_len[idx]
7         offset = caminos_offset[idx]
8         delta_tau = float32(Q) / float32(length)
9         for k in range(length):
10             r = caminos_flat[offset + k*2]
11             c = caminos_flat[offset + k*2 + 1]
12             cuda.atomic.add(pheromone, (r, c), delta_tau)

```

Listing 4: Fragmento del kernel de depósito en GPU

8. Visualización

8.1. Mapa inicial

Al inicio del experimento se muestran dos gráficas: el mapa del entorno con origen, destino y obstáculos; y la matriz de feromonas iniciales τ_0 .

8.2. Curva de convergencia

Tras ejecutar el ciclo principal se generan dos curvas:

- **Longitud del mejor camino por iteración**, con una línea de referencia en la distancia Manhattan (cota inferior teórica del camino óptimo).
- **Tasa de éxito** (% de hormigas que llegan al destino) por iteración.

8.3. Animación

La animación muestra el estado del experimento frame a frame con las siguientes capas:

Color	Significado
Negro $\rightarrow$ rojo $\rightarrow$ amarillo (inferno)	Intensidad de feromonas
Gris oscuro	Obstáculos
Verde brillante	Origen (madriguera)
Rojo brillante	Destino (fuente de comida)
Cian	Posiciones actuales de las hormigas
Amarillo	Mejor camino acumulado hasta ese frame

Cuadro 4: Leyenda de la animación del experimento.

El panel lateral muestra los parámetros del experimento, estadísticas de la iteración actual y una barra de progreso.

8.4. Resultado final

Al finalizar el experimento se producen dos visualizaciones estáticas: el mapa con el mejor camino encontrado trazado como línea, y el mapa de calor de feromonas final que evidencia los caminos más reforzados por la colonia.

9. Resultados obtenidos

La siguiente tabla muestra los resultados de una ejecución representativa:

Parámetro / Métrica	Valor
GPU utilizada	NVIDIA Tesla T4
Memoria VRAM	14.56 GB
Compute Capability	7.5
Mapa	21×18 celdas
Hormigas (N)	35
Iteraciones (M)	135
Mejor camino encontrado	11 pasos
Tiempo total de ejecución	6.97 segundos
Tasa de éxito promedio	$> 80\%$ de hormigas por iteración

Cuadro 5: Resultados de ejecución representativa del experimento ACO.

El algoritmo converge rápidamente: el mejor camino se encontró en la primera iteración y se mantuvo estable durante las iteraciones restantes, lo que indica que los parámetros de feromona y heurística permiten una exploración efectiva desde el inicio.

9.1. Nota sobre la advertencia de ocupación GPU

Durante la ejecución se muestra el siguiente aviso de Numba:

`NumbaPerformanceWarning: Grid size 4 will likely result in GPU under-utilization due to low occupancy.`

Esta advertencia es esperada y no indica un error. Ocurre porque la matriz del experimento ($\sim 20 \times 20$ celdas) es pequeña respecto a la capacidad de la T4 (2.560 núcleos CUDA), generando solo 4 bloques en el grid 2D. En aplicaciones con matrices de mayor tamaño (500×500 o más), la GPU operaría cerca de su capacidad máxima y la aceleración sería significativamente mayor.

10. Conclusiones

1. El algoritmo ACO implementado encuentra rutas eficientes en matrices con obstáculos, demostrando el principio de inteligencia emergente sin coordinación central.
2. El uso de **Numba CUDA** permite delegar a la GPU las operaciones más paralelizables del algoritmo: cálculo de heurística, evaporación, depósito de feromonas y precálculo de pesos. Esto libera a la CPU para la lógica de decisión de cada hormiga individual.
3. El **backtracking** aumenta significativamente la tasa de hormigas que alcanzan el destino, especialmente en mapas con densidad de obstáculos superior al 20 %.
4. El **piso de feromona** τ_{min} previene la extinción prematura de caminos alternativos, manteniendo diversidad en la exploración.
5. La limitación de ocupación GPU observada es inherente al tamaño del problema (matrices pequeñas). El diseño del código es correcto y escalaría eficientemente a matrices de mayor dimensión.
6. La elección de `matplotlib.animation` sobre `tkinter` es técnicamente justificada por las restricciones del entorno Colab (*headless*), y produce resultados equivalentes en términos de visualización interactiva.